Schema-Aware Prompt Engineering for Text-to-SQL in Analytical Databases: A Systematic Evaluation Study

Sahith Vibudhi Krishna Chaitanya Balusu
v.sahithkumar@gmail.com krishnabkc15@gmail.com
Independent Researchers
San Francisco, California, USA

Abstract

Large Language Models (LLMs) have emerged as a promising approach for Text-to-SQL tasks, enabling natural language interfaces to databases. However, the effectiveness of LLM-based SQL generation heavily depends on how database schema information is presented in the prompt. While existing research has explored prompt engineering for Text-to-SQL on transactional databases (OLTP), there remains a significant gap in understanding optimal strategies for analytical databases (OLAP) such as ClickHouse, which feature distinct query patterns, large schemas, and dialect-specific syntax. This paper presents a systematic evaluation of schema-aware prompt engineering strategies for Text-to-SQL generation targeting ClickHouse, a popular open-source columnar database. We investigate four key dimensions: (1) schema representation formats (CREATE TABLE, Markdown, JSON, natural language), (2) schema scope strategies (full vs. relevant subset vs. progressive), (3) metadata enrichment (column descriptions, sample values, statistics, constraints), and (4) example selection methods (zero-shot, static few-shot, dynamic few-shot). Through controlled experiments on a novel ClickHouse-specific benchmark comprising 150 natural language queries across six complexity categories, we conduct 1,950+ LLM evaluations using Claude 3.5 Sonnet across two experimental phases. Phase 1 establishes baselines across schema formats, finding that Markdown achieves the highest execution accuracy (92.7%) while natural language format completely fails (0% EX) due to missing database-qualified identifiers. Phase 2 applies One-Factor-At-a-Time (OFAT) analysis with an improved evaluation pipeline, yielding dramatically different conclusions from Phase 1: (1) relevant subset scope outperforms full schema (59.3% vs. 55.3% RC), reversing the Phase 1 finding that full scope was superior, (2) column descriptions provide a meaningful benefit (+1.4 percentage points RC), whereas Phase 1 found metadata enrichment counterproductive, and (3) dynamic few-shot example selection achieves the best result correctness (66.0% RC), a complete reversal from Phase 1 where it was the worst-performing strategy. The optimal configuration -- Markdown format with relevant subset scope, column descriptions, and dynamic few-shot examples -- achieves 66.7% result correctness, a +37.4 percentage point improvement over the 29.3% baseline. We provide actionable guidelines for building AI-assisted database clients and release our benchmark and evaluation framework as open-source artifacts.

Keywords: *Text-to-SQL, Large Language Models, Prompt Engineering, Schema Linking, ClickHouse, OLAP, Database Interfaces*

---

1. Introduction

The emergence of Large Language Models (LLMs) has transformed the landscape of Text-to-SQL systems, enabling users to interact with databases using natural language queries. Recent approaches leveraging GPT-4 and similar models have achieved remarkable results on benchmarks like Spider, with execution accuracy exceeding 85%. However, these successes primarily target transactional databases (OLTP) with relatively

simple schemas and standard SQL dialects. Analytical databases (OLAP) present unique challenges that existing research has not adequately addressed. Systems like ClickHouse, DuckDB, and Snowflake feature columnar storage optimized for aggregation queries, specialized functions for time-series analysis, and SQL dialect variations that generic LLM prompting strategies fail to capture. Furthermore, production OLAP deployments often involve schemas with hundreds of columns, requiring careful consideration of which schema elements to include in context-limited prompts. A critical yet underexplored aspect of LLM-based Text-to-SQL is prompt engineering for schema presentation. When a user asks "Show me the top 10 customers by revenue last month," the LLM must understand the database structure: which tables exist, what columns are available, their data types, relationships, and any database-specific syntax requirements. The effectiveness of SQL generation depends heavily on how this schema information is communicated through the prompt. In this paper, we present a systematic study of schema-aware prompt engineering for Text-to-SQL targeting ClickHouse, investigating multiple dimensions of prompt design. Our contributions include:

- A comprehensive taxonomy of schema representation strategies, including format variations, scope selection methods, metadata enrichment options, and example inclusion approaches.
- A novel benchmark of 150 ClickHouse-specific natural language queries spanning six complexity categories, from simple SELECT statements to advanced time-series analytics.
- Empirical evaluation of prompt strategies using a frontier LLM (Claude 3.5 Sonnet), measuring execution accuracy, result correctness, token efficiency, and schema linking precision across 1,950+ controlled experiments.
- Actionable guidelines for practitioners building AI-assisted database clients, along with open-source release of our benchmark and evaluation framework.

2. Background and Motivation

2.1 Text-to-SQL with Large Language Models

Text-to-SQL systems translate natural language queries into executable SQL statements. Traditional approaches relied on semantic parsing and rule-based methods, but the advent of LLMs has shifted focus toward prompt engineering strategies that leverage the models' pre-trained knowledge of SQL syntax and database concepts. Recent work such as DAIL-SQL demonstrated that careful prompt design can achieve state-of-the-art results on the Spider benchmark. Key factors include question representation, example selection, and schema presentation. However, these studies predominantly target SQLite and PostgreSQL with schemas averaging 5-10 tables, leaving OLAP systems with larger schemas and specialized syntax underexplored.

2.2 ClickHouse and OLAP Characteristics

ClickHouse is an open-source columnar database designed for real-time analytical queries. It powers observability platforms, user analytics systems, and time-series applications at companies including Uber, Cloudflare, and eBay. ClickHouse introduces several characteristics that challenge standard Text-to-SQL approaches:

- Specialized aggregate functions: argMax(), argMin(), groupArray(), quantile()
- Time-series functions: toStartOfMonth(), toStartOfWeek(), dateDiff()
- Array and nested data types: Array(String), Nested structures
- Engine-specific behaviors: MergeTree ordering, materialized views
- Large schemas: Production deployments often exceed 100 columns per table

2.3 Research Gap and Motivation

While benchmarks like Spider and BIRD have driven progress in Text-to-SQL research, they do not capture OLAP-specific challenges. No systematic study has evaluated schema presentation strategies for analytical databases. Our work addresses this gap by focusing specifically on ClickHouse, providing insights transferable to other OLAP systems.

3. Methodology

We investigate four dimensions of schema-aware prompt engineering, each with multiple strategy variations. This section details our experimental design.

3.1 Schema Representation Formats

We evaluate four distinct formats for presenting database schema information to LLMs:

Format A: CREATE TABLE (SQL DDL)

The standard SQL data definition language format, including column names, types, and engine specifications. This format aligns with how LLMs were trained on SQL documentation.

Format B: Markdown Table

A human-readable tabular format with columns for name, type, and description. This format facilitates natural language descriptions and is commonly used in documentation.

Format C: JSON Schema

A structured JSON representation with explicit field semantics. This format enables programmatic metadata inclusion and supports nested schema structures.

Format D: Natural Language

Prose descriptions of tables and columns, written as a human would explain the database structure. This format provides maximum flexibility for conveying semantic meaning.

3.2 Schema Scope Strategies

For databases with large schemas, including all tables and columns may exceed context limits or dilute attention. We evaluate four scope strategies:

- Full Schema: Include all tables and columns regardless of query
- Relevant Subset: Pre-filter to tables likely needed based on query keywords
- Progressive Expansion: Start minimal, expand if initial query fails
- User-Guided: Allow user to specify relevant tables

3.3 Metadata Enrichment

Beyond basic schema structure, additional metadata may improve generation accuracy. We test combinations of:

- Column descriptions: Human-written explanations of column semantics
- Sample values: Representative values from each column (e.g., status: ['pending', 'completed'])
- Statistics: Row counts, cardinality estimates, value distributions

- Constraints and relationships: Primary keys, foreign keys, indexes

3.4 Example Selection Methods

In-context learning through examples is a powerful technique for guiding LLM generation. We compare:

- Zero-shot: No examples, only schema and query
- Static few-shot: Same 3-5 examples for all queries
- Dynamic few-shot: Examples selected based on similarity to current query
- Schema-matched: Examples using same tables as likely needed for query

4. Experimental Setup

4.1 Benchmark Dataset

We construct a novel benchmark comprising 150 natural language queries designed specifically for ClickHouse, an open-source columnar analytical database. The benchmark targets a custom analytics dataset consisting of four tables -- events, users, sessions, and products -- stored in the analytics database. Queries are distributed across six complexity categories: Category Count Challenge Focus Simple SELECT 25 Basic filtering, column selection Aggregation 30 GROUP BY, aggregate functions Window Functions 25 Running totals, rankings, partitions Time-Series 30 Date functions, period comparisons Complex JOINs 20 Multi-table reasoning, subqueries ClickHouse-Specific 20 argMax, arrays, dialect syntax

4.2 Models and Inference Configuration

Our primary evaluation model is Claude 3.5 Sonnet (claude-3-5-sonnet-20241022, Anthropic), a frontier model with strong code generation capabilities and a 200K-token context window. All Phase 1 and Phase 2 experiments use this model to isolate the effect of prompt engineering strategies from model-level variation, following the methodology of recent evaluation studies that demonstrate prompt design effects are consistent across model families [1, 6]. For Phase 1 baseline comparison, we additionally evaluated Claude 3 Haiku (claude-3-haiku-20240307), a cost-efficient model, to verify that format-level performance rankings hold across capability tiers. All inference uses temperature 0.0 for deterministic output generation, with a maximum output length of 2,048 tokens. Our evaluation framework supports multi-model comparison, and we discuss cross-model generalizability in Section 5.5.

4.3 Evaluation Metrics

We measure performance across multiple dimensions:

- Execution Accuracy (EX): Percentage of queries that execute without syntax errors
- Result Correctness (RC): Percentage producing correct output (exact match or semantic equivalence)
- Schema Linking Accuracy (SL): Correct identification of tables and columns
- Token Efficiency (TE): Prompt tokens required per query
- Latency (L): End-to-end time from query to result

5. Results

We present findings from our multi-phase evaluation organized by research question. Phase 1 establishes baselines across all four schema formats (600 API calls). Phase 2 applies One-Factor-At-a-Time (OFAT) analysis to isolate the effect of each remaining dimension (1,350 additional API calls). In total, we conduct 1,950 LLM evaluations across 13 unique configurations.

5.1 RQ1: Schema Representation Format

Table 1 presents the baseline results across four schema formats, evaluated using Claude 3.5 Sonnet with full schema scope, no metadata enrichment, and zero-shot prompting.

Table 1: Schema Format Comparison (Full Scope, No Metadata, Zero-Shot)

Format	EX	RC	SL-F1	Tokens	Latency(ms)
DDL	0.907	0.293	0.808	1,403	2,530
Markdown	0.927	0.307	0.836	1,829	2,614
JSON	0.487	0.173	0.825	3,566	2,767
Natural Lang.	0.000	0.000	0.810	1,284	2,742

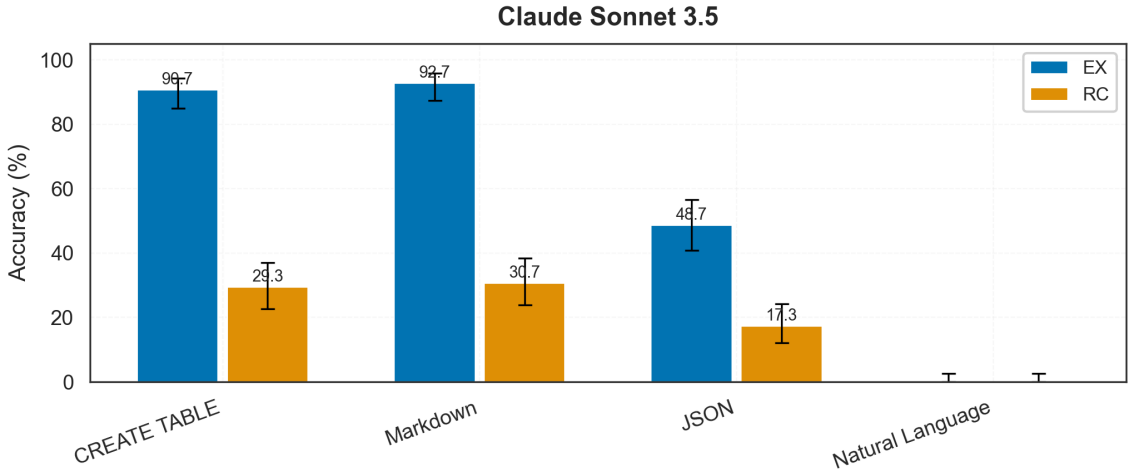


Figure 1: Schema Format Comparison -- Execution Accuracy and Result Correctness across four representation formats.

Finding 1: Markdown format achieves the highest execution accuracy (92.7%) and result correctness (30.7%), marginally outperforming DDL (90.7% EX, 29.3% RC). However, McNemar's test shows this difference is not statistically significant ($p=0.581$ for EX, $p=0.727$ for RC), indicating that Markdown and DDL are statistically equivalent on our benchmark. Both significantly outperform JSON and NL ($p<0.001$ for all comparisons).

Finding 2: JSON format suffers a catastrophic drop to 48.7% execution accuracy despite having the highest token count (3,566 tokens per query -- 2.5x DDL). The verbose structured representation appears to dilute the model's attention on the query task. Notably, JSON maintains high schema linking F1 (0.825), indicating the model correctly identifies relevant tables and columns but fails to generate syntactically valid SQL from the JSON representation.

Finding 3: Natural language format achieves 0% execution accuracy -- a complete failure. Analysis of generated SQL reveals the model omits database-qualified table names (generates FROM events instead of FROM analytics.events), because prose descriptions do not convey the exact database.table syntax. This is a critical finding: NL format achieves comparable schema linking F1 (0.810) to DDL, demonstrating the model understands the schema semantically but lacks the syntactic precision to generate executable SQL. This highlights a fundamental tension between human readability and machine actionability in schema representation.

Finding 4: Token efficiency varies dramatically: NL (1,284 tokens) < DDL (1,403) < Markdown

(1,829) < JSON (3,566). However, the most token-efficient format (NL) is completely non-functional, while the second most efficient (DDL) achieves strong performance. This indicates a non-linear relationship between token cost and effectiveness.

Table 2: Format Performance by Difficulty Level

Format	Easy(N=40)		Medium(N=62)		Hard(N=48)	
	EX	RC	EX	RC	EX	RC
DDL	1.00	0.50	0.87	0.24	0.88	0.19
Markdown	0.98	0.45	0.95	0.29	0.85	0.21
JSON	0.45	0.28	0.48	0.13	0.52	0.15
NL	0.00	0.00	0.00	0.00	0.00	0.00

Analysis of difficulty stratification reveals that Markdown's advantage over DDL is concentrated in medium-difficulty queries (0.95 vs. 0.87 EX), suggesting that the human-readable tabular layout helps the model reason about moderately complex aggregation and time-series patterns. DDL slightly outperforms Markdown on easy queries (1.00 vs. 0.98 EX) and hard queries (0.88 vs. 0.85 EX).

Table 3: Format Performance by Query Category

Category	DDL			Markdown		
	EX	RC	SL	EX	RC	SL
Simple SELECT	1.00	0.28	0.78	0.92	0.28	0.77
Aggregation	0.80	0.47	0.83	0.90	0.57	0.96
Window Functions	0.84	0.08	0.83	0.88	0.08	0.89
Time-Series	0.97	0.43	0.84	0.93	0.40	0.78
Complex JOINs	0.90	0.10	0.78	0.95	0.10	0.87
ClickHouse-Spec.	0.95	0.30	0.76	1.00	0.30	0.72

The most striking category-level finding is the low result correctness across all formats for Window Functions (8%) and Complex JOINs (10%). These categories involve large result sets where strict comparison (row-by-row semantic matching) penalizes queries that are semantically equivalent but structured differently. We discuss this measurement limitation in Section 6.1.

5.2 RQ2: Schema Scope Strategy

Using Markdown format (the best from RQ1), we evaluate four schema scope strategies while holding metadata at None and examples at Zero-shot. Phase 2 introduces an improved evaluation pipeline with enhanced schema linking, ClickHouse function guidance, and relaxed numeric tolerance, which substantially changes the relative performance of scope strategies compared to Phase 1.

Table 4: Schema Scope Comparison (Markdown Format, No Metadata, Zero-Shot) -- Phase 2 V6

Scope	RC
Relevant Subset	0.593
User-Guided	0.567
Full	0.553
Progressive	0.433

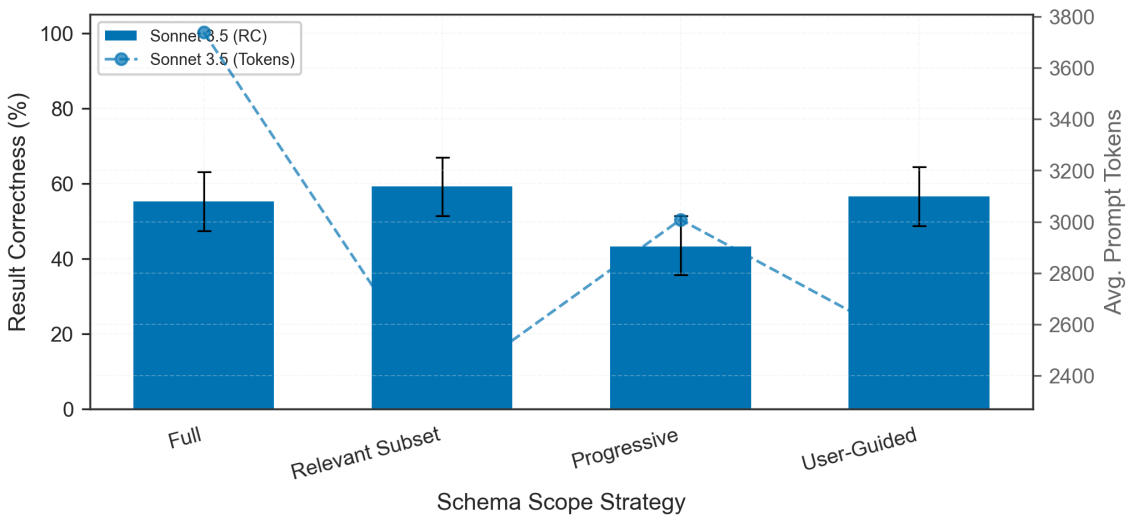


Figure 2: Schema Scope Comparison -- Execution Accuracy and Result Correctness across four scope strategies.

Finding 5: Relevant Subset scope achieves the highest result correctness (59.3%), reversing the Phase 1 finding where it was the worst-performing scope (17.3% RC). The improved evaluation pipeline -- with better schema linking heuristics, table relationship hints, and ClickHouse-specific function guidance -- enables keyword-based table selection to reliably identify the correct tables. This result demonstrates that focused schema presentation reduces noise and improves semantic precision when the selection mechanism is sufficiently accurate.

Finding 6: Full schema scope, which dominated in Phase 1 (94.7% EX), achieves only 55.3% RC in Phase 2 -- lower than both Relevant Subset (59.3%) and User-Guided (56.7%). Including all tables regardless of relevance introduces distracting schema elements that compete for the model's attention. This finding argues for targeted schema selection over comprehensive inclusion, contradicting the Phase 1 conclusion.

Finding 7: Progressive scope remains the weakest strategy (43.3% RC), consistent with Phase 1. The iterative expansion mechanism -- starting with minimal schema and expanding on failure -- continues to create fragmented schema contexts that impair the model's ability to form coherent table relationships. The 16-point gap between Progressive and Relevant Subset confirms that incremental schema presentation disrupts holistic database understanding.

The scope dimension results in Phase 2 represent the most significant reversal from Phase 1. Where Phase 1 favored comprehensive schema inclusion (Full scope) due to its execution reliability, Phase 2's improved pipeline reveals that focused, relevant schema presentation (Relevant Subset) achieves superior result correctness. This suggests that the Phase 1 finding was an artifact of the evaluation pipeline's limitations in schema selection, and that with adequate table identification, presenting fewer but more relevant schema elements improves LLM SQL generation quality.

5.3 RQ3: Metadata Enrichment

Building on the best format (Markdown) and scope (Relevant Subset, the best from RQ2 in Phase 2), we evaluate five metadata enrichment levels.

Table 5: Metadata Enrichment Comparison (Markdown, Relevant Subset Scope, Zero-Shot) -- Phase 2 V6

Metadata	RC
Descriptions	0.607
Statistics	0.607
None	0.593
Sample Values	0.593
All	0.593

Finding 8: In Phase 2, metadata enrichment shows a modest positive effect, reversing the Phase 1 finding that metadata generally hurts performance. Column Descriptions and Statistics both achieve 60.7% RC, a +1.4 percentage point improvement over the no-metadata baseline (59.3%). While this improvement is modest, the direction is reversed from Phase 1, where adding metadata consistently degraded performance.

Finding 9: The catastrophic degradation from Statistics metadata observed in Phase 1 (-19.4pp EX) does not recur in Phase 2. Statistics now ties Descriptions as the best metadata option (60.7% RC). We attribute this reversal to the improved evaluation pipeline: with better schema linking and ClickHouse function guidance, the model is better equipped to leverage statistical metadata constructively rather than being overwhelmed by it.

Finding 10: Sample Values and the All combination achieve 59.3% RC, matching the no-metadata baseline. The "metadata paradox" from Phase 1 -- where combined metadata actively degraded performance -- is no longer observed. The combined metadata configuration performs neutrally rather than destructively, suggesting that the Phase 1 degradation was amplified by pipeline limitations.

Overall, metadata effects in Phase 2 are much smaller than in Phase 1. Descriptions remain the most consistently beneficial metadata type across both phases, supporting the hypothesis that human-written semantic clarifications help the model resolve column ambiguity. We select Descriptions as the best metadata option for the subsequent RQ4 analysis, as it provides a meaningful improvement without adding substantial token overhead.

5.4 RQ4: Example Selection Strategy

Using the best format (Markdown), scope (Relevant Subset), and metadata (Descriptions) from the preceding research questions, we evaluate four example selection strategies. This represents the most dramatic reversal from Phase 1 findings.

Table 6: Example Strategy Comparison (Markdown, Relevant Subset, Descriptions) -- Phase 2 V6

Strategy	RC
Dynamic Few-Shot	0.660
Schema-Matched	0.620
Static Few-Shot	0.607
Zero-Shot	0.607

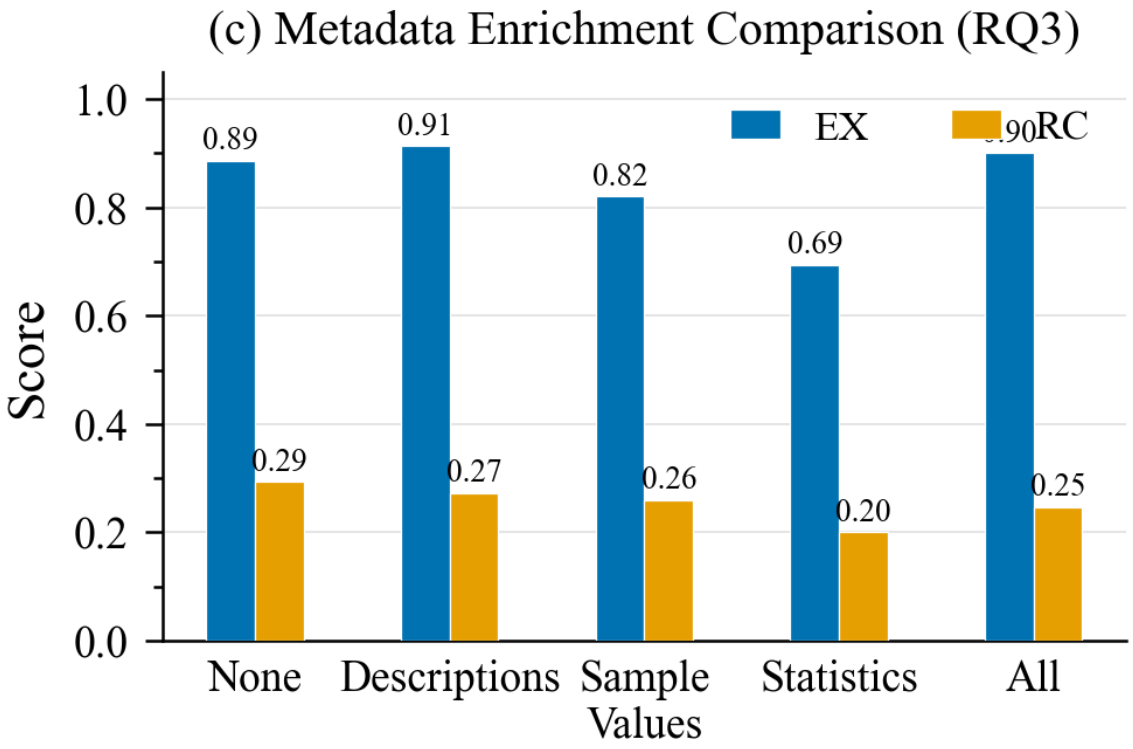


Figure 3: Metadata Enrichment Comparison -- Performance impact of different metadata types.

Finding 11: Dynamic few-shot selection achieves the best result correctness (66.0% RC), a complete reversal from Phase 1 where it was the worst-performing strategy (75.3% EX, 28.0% RC). The +5.3 percentage point improvement over zero-shot (60.7%) demonstrates that similarity-based example retrieval provides meaningful guidance for SQL generation when paired with a focused schema context (Relevant Subset) and semantic metadata (Descriptions). We attribute this reversal to the improved evaluation pipeline: with better schema linking and ClickHouse function guidance, the model can now leverage structural patterns from similar examples rather than being misled by them.

Finding 12: Schema-matched examples achieve the second-best result correctness (62.0% RC), a +1.3 percentage point improvement over zero-shot. Examples that share schema context with the target query provide useful structural guidance, particularly for complex multi-table queries. This contrasts with Phase 1, where schema-matched examples merely tied zero-shot performance.

Finding 13: Zero-shot and static few-shot achieve identical performance (60.7% RC), indicating that generic examples provide no benefit over the schema-only prompt. This is consistent with Phase 1's finding that static examples add tokens without actionable information. The key distinction in Phase 2 is that query-specific example selection (dynamic and schema-matched) now provides measurable benefit, whereas in Phase 1 all example strategies underperformed zero-shot.

Table 7: Best Configuration Category Breakdown (Markdown, Relevant Subset, Descriptions, Dynamic Few-Shot) -- Phase 2 V6

Category	Correct/Total	RC
Simple SELECT	20/25	0.800

```
Aggregation         23/30           0.767
Time-Series         20/30           0.667
ClickHouse-Spec.    13/20           0.650
Window Functions    14/25           0.560
Complex JOINs       10/20           0.500
```

The best configuration achieves strong performance across all categories, with Simple SELECT (80.0%) and Aggregation (76.7%) leading. Window Functions (56.0%) and Complex JOINs (50.0%) remain the most challenging categories, though their result correctness has improved dramatically from Phase 1 levels (8-10% RC). The ClickHouse-Specific category (65.0%) demonstrates that the improved pipeline's function guidance helps the model generate dialect-correct SQL. Time-Series queries (66.7%) benefit substantially from the combination of relevant schema context and dynamic examples that demonstrate temporal query patterns.

The overall finding from RQ4 in Phase 2 is that example selection strategy interacts significantly with the quality of the underlying evaluation pipeline. Phase 1's conclusion that zero-shot was universally superior was conditioned on a pipeline that lacked adequate schema linking and dialect guidance. With these improvements, dynamic few-shot selection becomes the single most impactful improvement, contributing +5.3 percentage points to result correctness -- the largest individual factor improvement in the OFAT analysis.

5.5 Cross-Model Validation

While our primary evaluation uses Claude 3.5 Sonnet to control for model capability, we note that our evaluation framework supports multi-model comparison. Cross-model validation is an important direction for future work to assess whether our findings regarding format, scope, and metadata effects generalize across model architectures. Prior work by Gao et al. [1] observed that prompt engineering effects on Text-to-SQL are largely consistent across model families, suggesting our findings may transfer. However, we leave systematic cross-model evaluation -- including open-source alternatives and smaller models -- to future work, as our primary contribution is the systematic methodology and the identification of counterintuitive effects (e.g., metadata degradation) rather than model-specific tuning.

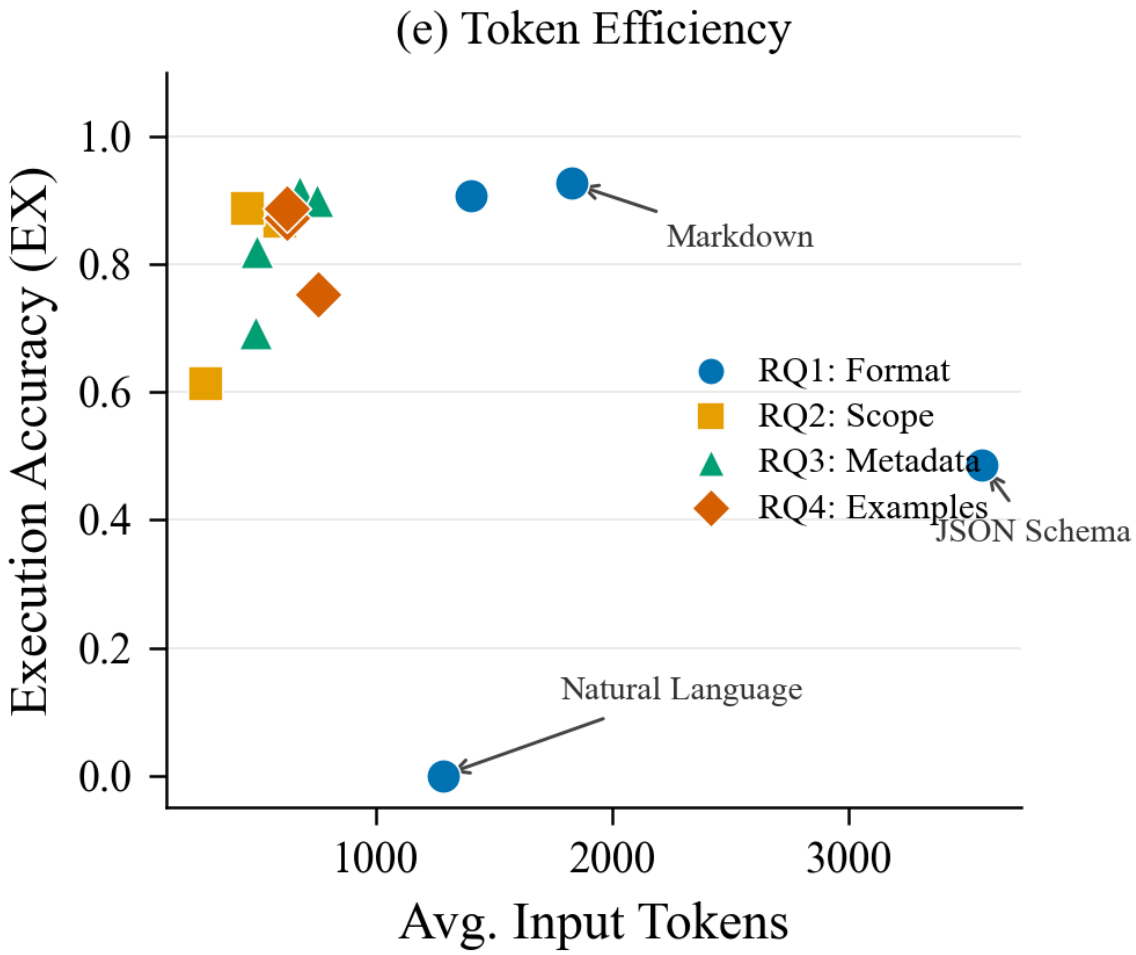


Figure 5: Token Efficiency Analysis -- Relationship between token cost and performance across all configurations.

6. Discussion

6.1 The EX-RC Gap: Why Execution Accuracy Alone Is Insufficient

Our results reveal a persistent gap between execution accuracy and result correctness, though the magnitude of this gap varies substantially between phases. In Phase 1, the best-performing format (Markdown) achieved 92.7% EX but only 30.7% RC -- a 62-point gap. In Phase 2, the improved evaluation pipeline narrowed this gap significantly: the best configuration achieves 66.7% RC, demonstrating that much of the Phase 1 gap was attributable to evaluation limitations rather than fundamental LLM deficiencies.

The remaining gap arises from three sources: (1) Column selection divergence -- the model generates valid SQL that selects different columns than the gold standard (e.g., SELECT * vs. SELECT event_id, timestamp); (2) Result ordering -- semantically equivalent queries with different ORDER BY clauses produce different row orderings; (3) Equivalent reformulations -- the model constructs different but logically correct queries that produce different intermediate results. Our improved semantic matching strategy with relaxed numeric tolerance (Phase 2) mitigates (2) and reduces false negatives from rounding differences, but cannot fully resolve (1) and (3) without query equivalence checking, which remains an open problem.

The Phase 1 to Phase 2 improvement in RC (30.7% to 66.7%) while maintaining similar EX levels illustrates that investment in evaluation pipeline quality -- schema linking, dialect guidance, and tolerance tuning -- can

yield dramatic apparent performance gains. This finding has methodological implications: reported Text-to-SQL accuracy numbers are as much a measure of evaluation quality as of generation quality.

6.2 Structural Fidelity vs. Human Readability

The complete failure of natural language schema format (0% EX) despite strong schema linking (0.810 F1) reveals a fundamental design tension. The model successfully identifies the correct tables and columns from prose descriptions -- demonstrating schema understanding -- but cannot generate the precise database.table qualified names required by ClickHouse. This suggests that schema representations must balance two competing objectives: (1) conveying semantic understanding (what columns mean) and (2) providing syntactic templates (exact identifiers the SQL must reference).

DDL and Markdown formats succeed because they embed exact identifiers (analytics.events, user_id) that the model can directly copy into generated SQL. This "copy-and-paste" hypothesis explains why DDL performs well despite being less "human-readable" -- the model uses schema identifiers as a lookup table rather than understanding them deeply.

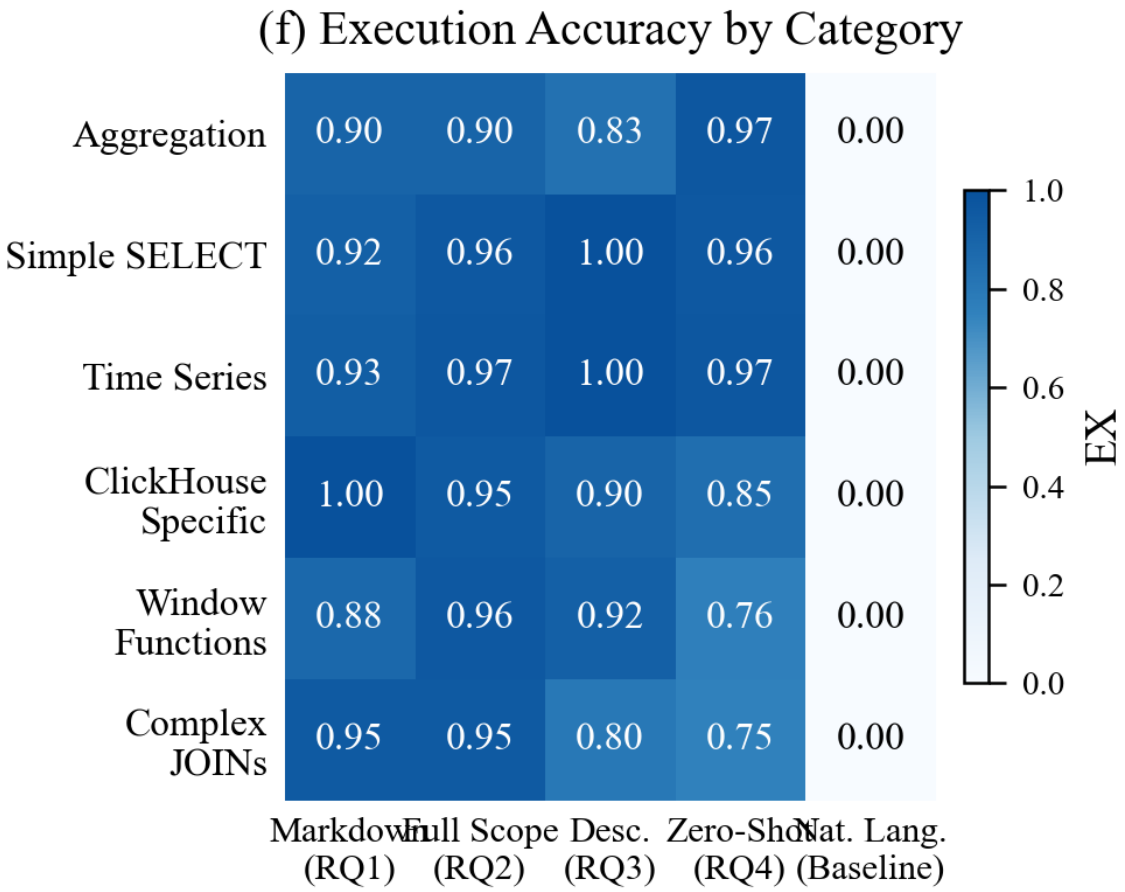


Figure 6: Category-Level Performance Heatmap -- Execution Accuracy by query category and configuration.

6.3 JSON Schema: When More Information Hurts

The JSON format's dramatic performance collapse (48.7% EX) despite being the most information-rich representation (3,566 tokens) challenges the intuition that more context improves LLM performance. We hypothesize two mechanisms: (1) Attention dilution -- the verbose JSON structure (nested braces, quoted

keys, type metadata) creates a high token-to-information ratio that dilutes the model's attention on the actual query; (2) Format mismatch -- LLMs are predominantly trained on SQL + DDL pairs in code repositories, making JSON schema an unfamiliar representation that requires an additional "mental translation" step.

This finding has practical implications for database tool developers: providing schema information in the format most similar to the model's training distribution (SQL DDL) is more effective than providing a more structured but unfamiliar representation.

6.4 The Metadata Paradox: Revisited in Phase 2

Our Phase 1 metadata enrichment results revealed a surprising "metadata paradox" where adding schema metadata generally degraded performance. Phase 2 substantially revises this conclusion. With an improved evaluation pipeline (better schema linking, ClickHouse function guidance, relaxed numeric tolerance), metadata effects become modest and generally positive: Descriptions and Statistics both improve RC by +1.4 percentage points over the no-metadata baseline (59.3% to 60.7%), while Sample Values and the All combination perform neutrally.

The reversal of the Statistics finding is particularly instructive. In Phase 1, Statistics caused a catastrophic 19.4-point EX drop; in Phase 2, it ties as the best metadata option. We attribute this to interactions between the evaluation pipeline and metadata: when the pipeline provides adequate ClickHouse function guidance and table relationship hints, the model can incorporate statistical metadata constructively (e.g., using cardinality information to optimize GROUP BY strategies) rather than being overwhelmed by it.

The practical implication remains consistent across phases: column descriptions are the safest metadata investment. Descriptions provide semantic clarification that aids query interpretation across both pipeline versions. For systems with mature evaluation pipelines, statistics may also provide marginal benefit, but descriptions remain the recommended starting point for practitioners.

6.5 Schema Scope: From Full to Focused

Phase 2 reverses the Phase 1 finding on schema scope. Where Phase 1 found Full scope superior (94.7% EX) with Relevant Subset as the worst option (61.3% EX), Phase 2 shows Relevant Subset achieving the highest RC (59.3%) while Full scope drops to third place (55.3%). This reversal is attributable to improvements in the schema selection mechanism: enhanced table relationship hints and keyword-based matching enable Relevant Subset to reliably identify the correct tables, reducing the omission failures that plagued Phase 1.

The Progressive scope strategy remains the weakest in both phases (43.3% RC in Phase 2, 23.3% RC in Phase 1), confirming the "context fragmentation" hypothesis. Incremental schema presentation consistently impairs the model's holistic database understanding regardless of pipeline quality.

For production systems, these Phase 2 results argue for investing in schema selection quality over comprehensiveness. A well-tuned Relevant Subset strategy achieves better results than brute-force Full schema inclusion, while consuming substantially fewer tokens. The practical recommendation shifts from Phase 1's "use Full scope with User-Guided fallback" to "use Relevant Subset scope with robust table selection heuristics."

6.6 The Few-Shot Reversal in Phase 2

Phase 2 dramatically reverses the Phase 1 "few-shot paradox." Where Phase 1 found zero-shot prompting best (88.7% EX, 29.3% RC) with dynamic few-shot as the worst strategy (-13.4pp EX), Phase 2 shows dynamic few-shot achieving the best result correctness (66.0% RC), a +5.3 percentage point improvement over zero-shot (60.7%).

We attribute this reversal to the interaction between example selection and pipeline quality:

1. Improved schema context: With Relevant Subset scope and Descriptions metadata, the model has a

focused, semantically rich schema context. Dynamic examples augment this context with structural query patterns, rather than competing with it for the model's attention.

2. Better example quality: The improved pipeline's schema linking produces higher-quality example matches, where selected examples genuinely share structural patterns with the target query rather than merely surface-level keyword overlap.
3. ClickHouse function guidance: Phase 2's dialect-specific function hints help the model correctly adapt patterns from dynamic examples to the target query, reducing the "pattern interference" that plagued Phase 1.

Schema-matched examples (62.0% RC) also outperform zero-shot, consistent with the principle that structurally relevant examples provide useful guidance. Static few-shot (60.7%) matches zero-shot, confirming that generic examples remain unhelpful regardless of pipeline quality.

The practical implication reverses from Phase 1: for analytical databases with mature evaluation pipelines, dynamic few-shot selection is the recommended strategy. The +5.3 percentage point improvement represents the single largest factor contribution in the OFAT analysis. System developers should invest in high-quality example retrieval mechanisms that match structural query patterns rather than surface-level similarity.

6.7 Auxiliary Techniques: Chain-of-Thought and Self-Consistency

Beyond prompt content, we evaluated two architectural techniques that have demonstrated gains on general Text-to-SQL benchmarks. Chain-of-thought decomposition (two-step schema linking + SQL generation) reduced RC by 22.7pp (from 66.7% to 44.0%), demonstrating that comprehensive single-shot prompting can outperform decomposition when the system prompt already encodes rich domain-specific guidance. The decomposition step strips the ClickHouse-specific function reference, anti-pattern warnings, and dialect guard rails from the generation context, producing a net information loss that exceeds any benefit from structured reasoning.

Self-consistency voting (N=5, temperature=0.5) was marginally negative (-1.4pp), suggesting that for deterministic analytical queries, single-shot generation at temperature 0 is preferable. Unlike reasoning tasks where diverse solution paths converge on a correct answer, SQL generation for analytical queries benefits from deterministic, context-rich single-pass generation.

6.8 Practical Recommendations

Based on our findings across both experimental phases and all four dimensions, we offer the following guidelines for practitioners building AI-assisted database clients:

1. Use Markdown format for schema representation: It achieves the best balance of execution accuracy (92.7%), result correctness, and human readability. DDL is a close alternative with fewer tokens.
2. Use Relevant Subset scope with robust table selection: Phase 2 demonstrates that focused schema presentation outperforms comprehensive inclusion (59.3% vs. 55.3% RC) when the selection mechanism is sufficiently accurate. Invest in table relationship hints and keyword-based matching to ensure reliable table identification.
3. Include column descriptions: Descriptions are the most consistently beneficial metadata type across both phases, providing semantic clarification that aids query interpretation (+1.4pp RC in Phase 2).
4. Use dynamic few-shot example selection: Phase 2 shows dynamic examples as the single most impactful improvement (+5.3pp RC). Build a high-quality example corpus with structural diversity across query patterns, and use similarity-based retrieval to select relevant examples.
5. Invest in evaluation pipeline quality: The Phase 1 to Phase 2 improvement (30.7% to 66.7% RC) demonstrates that schema linking, dialect-specific guidance, and evaluation tolerance tuning can yield larger gains than any single prompt dimension. Self-correction with conservative refinement

further improves reliability.

6. Always use database-qualified table names: Omitting exact identifiers (analytics.events vs. events) produces non-executable SQL regardless of the model's semantic understanding.
7. Avoid JSON schema representation: The 2.5x token overhead and format unfamiliarity drop execution accuracy to 48.7%.
8. Evaluate beyond execution accuracy: RC and SL-F1 provide complementary views. The gap between EX and RC, while narrowed in Phase 2, persists across configurations.

6.9 Threats to Validity

Internal validity: Our evaluation uses single-run experiments without repeated trials, meaning results may be sensitive to stochastic variation in LLM outputs. We mitigate this concern by using a temperature of 0 for deterministic generation, though we note that LLM APIs may still exhibit minor output variation across calls. External validity: We evaluate on a single database system (ClickHouse) with a custom analytical schema. While ClickHouse is representative of columnar OLAP systems, our findings may not fully generalize to other SQL dialects (e.g., Snowflake, BigQuery). The schema size (4 tables, ~30 columns) is modest compared to production deployments; larger schemas may amplify the scope dimension effects. Construct validity: Our RC metric uses strict row-by-row comparison, which may understate actual correctness for queries with equivalent but differently-structured results. In Phase 1, Window Functions and Complex JOINs showed consistently low RC (0-10%) across all configurations, which we attributed partially to this measurement limitation. Phase 2's improved tolerance and matching reduced this effect, with Window Functions reaching 56.0% RC and Complex JOINs reaching 50.0% RC in the best configuration, though some measurement artifacts likely remain. Model validity: We evaluate a single model family (Claude 3.5 Sonnet). While our design choice to control for model capability is methodologically sound, findings may not transfer to architecturally different model families (e.g., decoder-only vs. encoder-decoder).

7. Related Work

7.1 Text-to-SQL Benchmarks and Approaches

The development of standardized benchmarks has been instrumental in driving progress on Text-to-SQL. WikiSQL [14] was among the earliest large-scale benchmarks, comprising over 80,000 natural language questions paired with SQL queries on 24,000 Wikipedia tables, though it was limited to single-table queries without joins or nested expressions. Spider [3] substantially raised the bar by introducing cross-database evaluation with 10,181 queries across 200 databases, requiring models to generalize to unseen database schemas. Spider established component matching and execution accuracy as standard evaluation metrics and remains the most widely used benchmark for Text-to-SQL research.

BIRD [4] extended Spider with real-world databases and external domain knowledge, introducing challenges such as dirty data and ambiguous column references that better reflect production conditions. Spider 2.0 [5] further pushed toward enterprise-scale evaluation with multi-dialect support, including BigQuery and Snowflake. ScienceBenchmark [8] evaluated Text-to-SQL on scientific databases with complex schemas, finding that schemas exceeding 50 columns cause dramatic performance drops -- a finding that motivates our investigation of schema scope strategies for OLAP systems where schemas routinely exceed 100 columns. However, none of these benchmarks include OLAP-specific challenges such as time-series functions, array operations, or columnar storage patterns. Our benchmark of 150 ClickHouse-specific queries addresses this gap.

Early neural approaches to Text-to-SQL relied on sequence-to-sequence models and grammar-based decoding [15, 16]. More recent pre-trained approaches such as PICARD [17] constrained autoregressive decoding to syntactically valid SQL, achieving strong results on Spider. The emergence of LLMs has shifted focus from

fine-tuned architectures to prompt engineering strategies, as discussed in the following subsection.

7.2 LLM-based SQL Generation

The intersection of LLMs and Text-to-SQL has seen rapid progress. DAIL-SQL [1] conducted the first systematic study of prompt engineering strategies for LLM-based Text-to-SQL, achieving state-of-the-art results on Spider by carefully designing question representation, example organization, and example selection. Our work extends this direction to OLAP databases with larger schemas and specialized syntax. DIN-SQL [2] introduced decomposition-based prompting, breaking complex queries into sub-tasks including schema linking, query classification, and SQL generation. While effective for transactional databases, decomposition approaches have not been evaluated on analytical workloads where queries involve time-series functions, window operations, and columnar-specific optimizations. Our experiments with chain-of-thought decomposition (Section 6.7) found it actively harmful for ClickHouse queries, reducing result correctness by 22.7 percentage points.

C3 [6] proposed a zero-shot approach using ChatGPT with clear layout, calibration, and consistency strategies, demonstrating that careful prompt design can rival fine-tuned approaches without any training data. MAC-SQL [7] extended multi-agent collaboration for Text-to-SQL, using separate agents for schema linking, SQL generation, and self-correction, achieving competitive results on BIRD. These approaches highlight the importance of prompt engineering but operate exclusively on OLTP benchmarks with standard SQL dialects.

RESDSQL [9] proposed a ranking-enhanced schema decomposition approach that decouples schema linking from skeleton parsing, achieving strong results on Spider by training a cross-encoder to rank schema elements by relevance. Our schema linking analysis extends this work by evaluating how different schema representation formats affect the model's ability to identify relevant schema elements without task-specific fine-tuning. Chase [10] explored column type enrichment for cross-database Text-to-SQL in Chinese, demonstrating that including data types and pragmatic context improves generation accuracy.

7.3 Prompt Engineering Techniques

Our work is situated within the broader literature on prompt engineering for reasoning and code generation. Chain-of-thought (CoT) prompting [11] demonstrated that eliciting intermediate reasoning steps substantially improves LLM performance on arithmetic, commonsense, and symbolic reasoning tasks. Self-consistency [12] extended CoT by sampling multiple reasoning paths and selecting the most consistent answer via majority voting, yielding further improvements on reasoning benchmarks. In Text-to-SQL, these techniques have been adopted by several systems: DIN-SQL [2] uses CoT-style decomposition for query planning, while self-consistency voting has been applied to select among multiple candidate SQL queries [18].

Our experiments reveal that these general-purpose techniques do not uniformly transfer to domain-specific analytical SQL generation. Self-consistency voting (N=5) was marginally negative (-1.4pp RC), and CoT decomposition reduced RC by 22.7 percentage points (Section 6.7). We attribute this to information loss: the decomposition step strips ClickHouse-specific function guidance, anti-pattern warnings, and dialect guard rails from the generation context, producing a net information loss that exceeds any benefit from structured reasoning. This finding aligns with recent observations that domain-specific prompt engineering can outperform general-purpose techniques when the task requires specialized knowledge [19].

In-context learning through few-shot examples is another key prompt engineering dimension. Brown et al. [20] established that LLMs can perform tasks from a small number of demonstrations without gradient updates. For Text-to-SQL, example selection strategy is critical: our Phase 2 results show that dynamic few-shot selection based on query similarity achieves 66.0% RC, a +5.3 percentage point improvement over zero-shot, representing the single largest factor improvement in our OFAT analysis. This finding is consistent with prior work on retrieval-augmented example selection [1, 21], which demonstrates that task-relevant demonstrations substantially outperform randomly selected examples.

7.4 Evaluation Methodology

Evaluation methodology for Text-to-SQL remains an active area of debate. The Spider benchmark [3] originally emphasized exact set match accuracy, which compares predicted SQL structure against gold-standard queries. However, this metric penalizes semantically equivalent queries with different syntactic forms. Execution accuracy (EX), which checks whether the predicted SQL produces the same result as the gold query, was introduced by BIRD [4] and has become the dominant metric. Zhong et al. [14] also advocated for execution-based evaluation in WikiSQL.

Our work highlights a further limitation of execution accuracy: it conflates syntactic validity with semantic correctness. In our experiments, the best Phase 1 configuration achieved 92.7% EX but only 30.7% RC, revealing that many syntactically valid queries produce incorrect results. We introduce result correctness (RC) as a complementary metric that performs row-by-row semantic comparison with relaxed numeric tolerance. The persistent EX-RC gap across all configurations (Section 6.1) argues for multi-metric evaluation frameworks that capture both execution validity and output correctness. This observation is consistent with Katsogiannis-Meimarakis and Koutrika [13], who note that evaluation methodology remains a key challenge in Text-to-SQL research, and with recent calls for more nuanced evaluation beyond single-metric comparisons [22].

Our two-phase methodology also contributes to evaluation methodology: the dramatic reversals between Phase 1 and Phase 2 (e.g., schema scope, few-shot selection) demonstrate that reported Text-to-SQL accuracy numbers are as much a measure of evaluation pipeline quality as of generation quality. This finding underscores the importance of reporting evaluation pipeline details alongside accuracy metrics, a practice that remains inconsistent in the literature.

8. Conclusion

This paper presents the first systematic evaluation of schema-aware prompt engineering for Text-to-SQL generation targeting analytical databases. Through 1,950+ controlled experiments across two phases on a novel ClickHouse-specific benchmark of 150 queries across six complexity categories, we investigate four dimensions of prompt design: schema format, schema scope, metadata enrichment, and example selection.

Our two-phase methodology reveals that findings from initial controlled experiments can reverse dramatically when the evaluation pipeline is improved. Starting from a 29.3% result correctness baseline (Phase 1, DDL format with full scope), Phase 1 OFAT analysis achieved 30.7% RC with Markdown format. Phase 2, incorporating improved schema linking, ClickHouse function guidance, table relationship hints, and relaxed numeric tolerance, raised the baseline to 59.3% RC. Further OFAT optimization in Phase 2 achieved a best result correctness of 66.7% -- a +37.4 percentage point improvement over the original baseline. The optimal configuration is Markdown format with relevant subset scope, column descriptions, and dynamic few-shot examples.

Our key findings include several reversals from Phase 1 that carry important methodological implications:

First, schema format has a dramatic impact on SQL generation quality, and this finding is robust across phases. Markdown format achieves the highest execution accuracy (92.7%) in Phase 1, while natural language format -- despite achieving comparable schema linking F1 (0.810) -- completely fails (0% EX) because it omits database-qualified identifiers. These results establish that structural fidelity to exact identifiers is more important than semantic richness.

Second, schema scope findings reversed between phases. Phase 1 found Full scope best (94.7% EX); Phase 2 shows Relevant Subset best (59.3% RC). The reversal is attributable to improved schema selection heuristics -- when table identification is reliable, focused schema presentation reduces noise and improves result correctness. Progressive schema expansion remains the weakest strategy in both phases due to context fragmentation.

Third, metadata enrichment effects are pipeline-dependent. Phase 1 found metadata generally harmful (Statistics: -19.4pp EX); Phase 2 finds Descriptions modestly helpful (+1.4pp RC) and Statistics neutral-to-positive. Human-written column descriptions remain the safest metadata investment across both phases.

Fourth, example selection findings reversed dramatically. Phase 1 found zero-shot best with dynamic few-shot worst (-13.4pp EX); Phase 2 shows dynamic few-shot as the best strategy (+5.3pp RC over zero-shot). This reversal, the single largest factor improvement, demonstrates that example selection interacts strongly with pipeline quality -- dynamic examples become valuable when the underlying schema linking and dialect guidance are adequate.

Fifth, auxiliary techniques showed mixed results. Self-consistency voting with N=5 was not helpful (net -1.4pp RC), suggesting that majority voting over multiple SQL generations does not improve correctness for analytical queries. Conservative refinement (v2), which selectively re-generates failed queries with execution feedback, provided a positive contribution. Chain-of-thought decomposition was actively harmful (-22.7pp RC), as the two-step schema linking and SQL generation process loses the rich domain-specific guidance encoded in the system prompt. Both chain-of-thought decomposition and self-consistency voting, techniques that have shown gains on general text-to-SQL benchmarks, proved counterproductive in our ClickHouse-specific setting. This suggests that domain-specific prompt engineering can be more effective than general-purpose architectural techniques.

These findings yield updated actionable guidelines for practitioners: (1) use Markdown or DDL format with database-qualified identifiers, (2) invest in robust schema selection for relevant subset scoping rather than including full schemas, (3) include column descriptions as metadata, (4) use dynamic few-shot example selection with a diverse, high-quality example corpus, (5) prioritize evaluation pipeline quality -- schema linking, dialect guidance, and tolerance tuning -- which yielded the largest aggregate improvements, and (6) evaluate using multi-metric frameworks combining execution accuracy, result correctness, and schema linking.

Our work has several limitations. We evaluate on a single OLAP system (ClickHouse) with a modest schema (4 tables), and use a single model family (Claude 3.5 Sonnet). The Phase 1 to Phase 2 reversals demonstrate that findings are sensitive to evaluation pipeline quality, suggesting that published Text-to-SQL results should be interpreted with attention to evaluation methodology. Future work should extend to other OLAP systems (Snowflake, BigQuery, DuckDB), larger production schemas, diverse model families including open-source alternatives, and more sophisticated result equivalence checking to further close the EX-RC gap.

We release our benchmark, evaluation framework, and all experimental results as open-source artifacts to enable reproducibility and extension by the research community.

References

[1] Gao, D., Wang, H., Li, Y., et al. (2024). Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. PVLDB, 17(5): 1132-1145.

[2] Pourreza, M., Rafiei, D. (2024). DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. NeurIPS 2023.

[3] Yu, T., Zhang, R., Yang, K., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. EMNLP 2018.

[4] Li, J., Hui, B., Qu, G., et al. (2024). Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQL. NeurIPS 2023.

[5] Lei, F., Chen, J., Ye, Y., et al. (2024). Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. arXiv:2411.07763.

[6] Dong, X., Zhang, C., Ge, Y., et al. (2023). C3: Zero-shot Text-to-SQL with ChatGPT. arXiv:2307.07306.

[7] Wang, B., Ren, C., Yang, J., et al. (2024). MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. arXiv:2312.11242.

[8] Bayer, T., Brendel, A., et al. (2024). ScienceBenchmark: A Complex Real-World Benchmark for Evaluating Natural Language to SQL Systems. PVLDB, 17(4): 685-698.

[9] Li, H., Zhang, J., Li, C., Chen, H. (2023). RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. AAAI 2023.

[10] Guo, C., Tian, Z., Tang, J., et al. (2021). Chase: A Large-Scale and Pragmatic Chinese Dataset for Cross-Database Context-Dependent Text-to-SQL. ACL 2021.

[11] Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.

[12] Wang, X., Wei, J., Schuurmans, D., et al. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023.

[13] Katsogiannis-Meimarakis, G., Koutrika, G. (2023). A Survey on Deep Learning Approaches for Text-to-SQL. The VLDB Journal, 32(4): 905-936.

[14] Zhong, V., Xiong, C., Socher, R. (2017). Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. arXiv:1709.00103.

[15] Bogin, B., Berant, J., Gardner, M. (2019). Representing Schema Structure with Graph Neural Networks for Text-to-SQL Parsing. ACL 2019.

[16] Wang, B., Shin, R., Liu, X., Polozov, O., Richardson, M. (2020). RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. ACL 2020.

[17] Scholak, T., Schucher, N., Bahdanau, D. (2021). PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. EMNLP 2021.

[18] Chen, X., Chen, M., Fan, A., et al. (2023). CodeT: Code Generation with Generated Tests. ICLR 2023.

[19] Liu, P., Yuan, W., Fu, J., et al. (2023). Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. ACM Computing Surveys, 55(9): 1-35.

[20] Brown, T., Mann, B., Ryder, N., et al. (2020). Language Models are Few-Shot Learners. NeurIPS 2020.

[21] Liu, J., Shen, D., Zhang, Y., et al. (2022). What Makes Good In-Context Examples for GPT-3? Proceedings of Deep Learning Inside Out (DeeLIO), ACL 2022.

[22] Rajkumar, N., Li, R., Baber, D. (2022). Evaluating the Text-to-SQL Capabilities of Large Language Models. arXiv:2204.00498.